

# Chapter 33. The Computer Science You Actually Need

Big-O from measured times, the structures that matter, trees and graphs, searching, sorting, recursion, hashing, and what interviews test.

Draft chapter

# Contents

## Chapter 33. The Computer Science You Actually Need

Why this matters

In plain English

33.1 Big-O: how work grows

33.2 The four structures you'll use every day

33.3 Stacks and queues

33.4 Trees and graphs

33.5 Searching and sorting

33.6 Recursion

33.7 Hashing, and why dictionaries are instant

33.8 Memoization, dynamic programming, and greedy

33.9 Memory, and the other costs

33.10 What coding interviews actually test

Common mistakes and how to spot them

In the real world: the invoice match that ran all afternoon

Tools

The project: make the matching script fast, and prove it

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

# Chapter 33. The Computer Science You Actually Need

Part III — Advanced Analytics & Analytics Engineering

## Chapter at a glance

**You will learn to:** describe how long an algorithm takes in the language everyone uses (Big-O), and see the curves in measured times · choose between a list, a dictionary, a set, and a tuple for a reason · use stacks and queues where they fit, including `deque` · walk trees and graphs with depth-first and breadth-first search, and detect cycles · tell linear search from binary search, and know what `sorted()` costs · write and read recursion without fear, and know when a loop is clearer · explain why a dictionary lookup is instant, and why the same trick powers a database hash join · recognize memoization, dynamic programming, and greedy algorithms when you meet them · think about memory as well as speed · and talk through a coding problem the way interviewers expect.

**Before you start:** Chapter 17 (Python: lists, dictionaries, loops, functions) and Chapter 29 (packaging and tests). Chapter 28's indexes and hash joins are the database version of several ideas here.

**Time needed:** 12–16 hours, spread over two to three weeks.

**Tools:** Python 3.12 and its standard library. Nothing else.

**Practice data:** `generate_ch33_data.py` in the companion folder builds 176,110 Riverstone order lines and 20,000 supplier invoice lines, which the chapter's project matches. Every timing shown was measured on the author's test machine (1 vCPU, 3 GB of memory, Ubuntu 24.04) and is reproduced by `ch33_timings.py`.

---

## Why this matters

You can have a long, well-paid career in data without knowing what a binary tree is. You cannot have one without meeting these three situations:

- **The script that got slower.** It ran in twenty seconds on last month's file. This month's file is twice the size and it's been going for forty minutes. Nothing changed except the data.
- **The interview.** Analytics engineering and data engineering interviews include a coding round, and the questions are all from this chapter's vocabulary: "can you do this without a nested loop?", "what's the complexity of your solution?"
- **The conversation with engineers.** When a backend engineer says "that endpoint is  $O(n^2)$  in the number of orders", you either know what that sentence claims or you nod.

This chapter is not a computer-science course. It's the subset that changes how you write Python next week, with every claim measured rather than asserted. One idea in it, the dictionary index in section 33.3, is worth the whole chapter: it turns the project's 74-second matching job into 0.07 seconds.

---

## In plain English

---

**Complexity is about how the work grows, not how long it takes today.**

Imagine finding one person in a hotel:

- You know the room number: walk there. One step, however big the hotel is. That's **constant time**.
- You must knock on every door: a hundred doors, a hundred knocks; a thousand doors, a thousand knocks. **Linear time**.
- The rooms are sorted and you can ask "higher or lower": a thousand rooms takes about ten questions, a million takes about twenty. **Logarithmic time**, and it barely notices when the hotel grows.
- You must introduce every guest to every other guest: a hundred guests is 4,950 introductions; a thousand is half a million. **Quadratic time**, and it's the reason most slow scripts are slow.

The point isn't the maths. It's that the hotel always gets bigger. A method that grows quadratically works fine on today's data, and the day the file doubles it takes four times as long. The way out is usually to build an index first, which is what a dictionary is, and what the database does when it builds a hash table for a join (Chapter 28).

---

## 33.1 Big-O: how work grows

---

**Big-O** describes how an algorithm's work grows with the size of its input, ignoring constants and small terms. It's a shape, not a stopwatch.

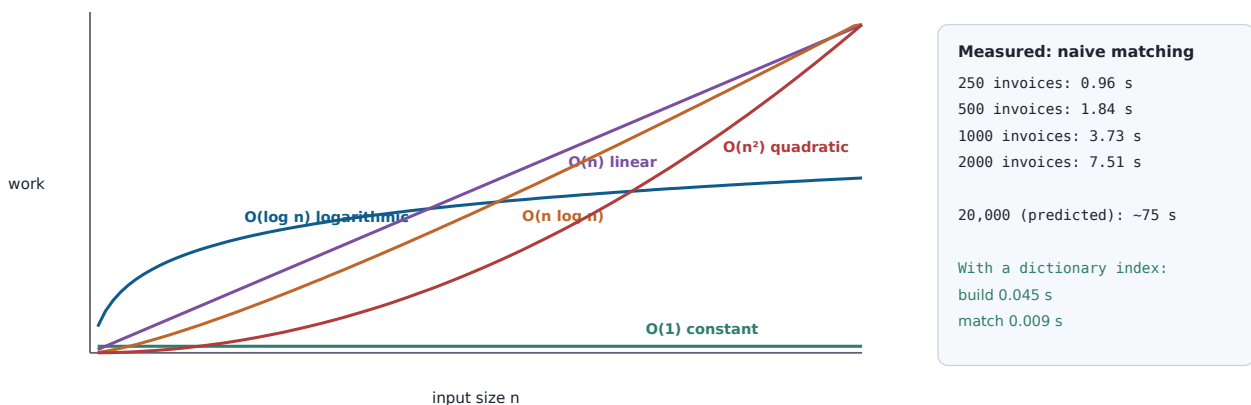
| Notation      | Called       | Work when the input doubles | Everyday example                          |
|---------------|--------------|-----------------------------|-------------------------------------------|
| $O(1)$        | constant     | unchanged                   | dictionary lookup, appending to a list    |
| $O(\log n)$   | logarithmic  | one more step               | binary search, index lookup in a database |
| $O(n)$        | linear       | doubles                     | scanning a list, reading a file           |
| $O(n \log n)$ | linearithmic | slightly more than doubles  | sorting                                   |
| $O(n^2)$      | quadratic    | four times                  | a loop inside a loop over the same data   |
| $O(2^n)$      | exponential  | squares                     | trying every combination                  |

Here is the shape that matters most, measured. The naive matching script (section 33.11's project) compares every invoice against every order line:

```
# terminal, in companion/ch33 (median of the runs in ch33_timings.py)
250 invoices x 176,110 order lines: 0.96 s
500 invoices x 176,110 order lines: 1.84 s
1000 invoices x 176,110 order lines: 3.73 s
2000 invoices x 176,110 order lines: 7.51 s
```

Double the invoices, double the time: that's linear *in the number of invoices*, because the order lines stayed the same. The quadratic part appears when both grow together, which is what happens in real life: next year Riverstone has twice the invoices **and** twice the orders, so the job takes four times as long. Extrapolating the last row, the full 20,000-invoice file takes about **75 seconds** today, and about five minutes when the business doubles.

How the work grows: the same five shapes, drawn to scale



Doubling the input doubles a linear algorithm's work and quadruples a quadratic one. The curves are drawn; the numbers on the right were measured.

Figure 33.1 – The shapes, from measured times. Constant and logarithmic barely move; quadratic leaves the page.

### How to read your own code's complexity, without any algebra:

1. **Count the loops over your data.** One loop over  $n$  items is  $O(n)$ . A loop inside a loop over the same  $n$  is  $O(n^2)$ .
2. **A lookup inside a loop is the thing to watch.** `if x in some_list` is itself a loop, so putting it inside a loop makes the whole thing quadratic. `if x in some_set` is not.
3. **Sorting is  $O(n \log n)$** , which is close enough to linear that sorting once is almost always worth it.
4. **Constants don't matter to Big-O, but they matter to you.**  $O(n)$  that reads from disk can be slower than  $O(n^2)$  in memory, for small  $n$ . Big-O tells you what happens as  $n$  grows; the stopwatch tells you what happens today.

**Watch out: Big-O ignores exactly the things that make code fast.** Two  $O(n)$  algorithms can differ by a factor of fifty. pandas and NumPy are fast not because their complexity is better but because their constants are tiny: the work happens in compiled code over contiguous memory instead of in a Python loop. Use Big-O to choose the approach, and a measurement to choose between two implementations of it.

## 33.2 The four structures you'll use every day

| Structure    | Written   | Keeps order?    | Lookup by value | Good at                               | Bad at                           |
|--------------|-----------|-----------------|-----------------|---------------------------------------|----------------------------------|
| <b>list</b>  | [1, 2, 3] | yes             | O(n)            | keeping a sequence, appending         | finding, removing from the front |
| <b>dict</b>  | {"a": 1}  | insertion order | O(1) by key     | lookups, grouping, counting           | nothing much                     |
| <b>set</b>   | {1, 2, 3} | no              | O(1)            | membership, deduplication, difference | anything ordered                 |
| <b>tuple</b> | (1, 2)    | yes             | O(n)            | fixed records, dictionary keys        | changing anything                |

The measured difference, on the project's data (176,110 keys, 1,000 lookups):

```
# terminal, in companion/ch33 (from ch33_timings.py)
list "in" : 1.451 s
set "in" : 0.00004 s
dict "in" : 0.00009 s
```

That's not a small improvement, it's four orders of magnitude, and it comes from one keystroke: `set(...)` instead of `list(...)`. The list must compare every element until it finds a match; the set computes a hash and goes straight there (section 33.7).

### The dictionary index, which is most of this chapter's value

Any time you find yourself looking something up inside a loop, **build a dictionary first**:

```
import csv

def load(path):
    with open(path, newline="", encoding="utf-8") as f:
        return list(csv.DictReader(f))

order_lines = load("match_data/order_lines.csv")
invoices = load("match_data/supplier_invoices.csv")
print(f"{len(order_lines):,} order lines, {len(invoices):,} invoice lines")

# the index: one pass over the order lines, keyed by what the invoice knows
index = {(line["order_id"], line["product_id"]): line for line in order_lines}
print(f"index holds {len(index):,} keys")

matched = sum(1 for invoice in invoices
              if (invoice["order_id"], invoice["product_id"]) in index)
print(f"matched {matched:,}, unmatched {len(invoices) - matched:,}")
```

```
176,110 order lines, 20,000 invoice lines
index holds 176,110 keys
matched 19,500, unmatched 500
```

One pass to build the index, one lookup per invoice: **O(n + m)** instead of **O(n × m)**. Measured, that's 0.045 seconds to build and 0.009 seconds to match all 20,000 invoices, against 75 seconds for the nested loop.

The same trick appears everywhere once you see it:

| Instead of                                                              | Do this                                                   |
|-------------------------------------------------------------------------|-----------------------------------------------------------|
| <code>for row in rows: if row["id"] in id_list</code>                   | <code>make id_set = set(id_list) first</code>             |
| <code>for a in table_a: for b in table_b: if a.key == b.key</code>      | <code>index table_b by key, then loop table_a once</code> |
| <code>counting with if key in counts: counts[key] += 1 else: ...</code> | <code>collections.Counter, or defaultdict(int)</code>     |
| <code>looking up a customer's name inside a loop</code>                 | <code>one dictionary built before the loop</code>         |

And the database does exactly this: Chapter 28's **hash join** builds a hash table of the smaller table, then scans the larger one once. When you build a dictionary index in Python, you are hand-writing a hash join.

## Grouping, counting, and the standard library

```
import csv
from collections import Counter, defaultdict

def load(path):
    # the same helper as above
    with open(path, newline="", encoding="utf-8") as f:
        return list(csv.DictReader(f))

order_lines = load("match_data/order_lines.csv")
invoices = load("match_data/supplier_invoices.csv")

by_product = defaultdict(list)
for line in order_lines:
    by_product[line["product_id"]].append(float(line["net_revenue"]))

revenue = {product: round(sum(values)) for product, values in by_product.items()}
for product in sorted(revenue, key=revenue.get, reverse=True)[:3]:
    print(f"product {product}: {len(by_product[product]):,} lines, revenue {revenue[product]:,}")

statuses = Counter(invoice["product_id"] for invoice in invoices)
print("most invoiced products:", statuses.most_common(3))
```

```
product 106: 22,084 lines, revenue 563,741,550
product 108: 22,042 lines, revenue 556,953,300
product 101: 22,075 lines, revenue 555,084,450
most invoiced products: [('101', 2607), ('107', 2561), ('104', 2514)]
```

`defaultdict(list)` removes the "is this key there yet?" dance, and `Counter` does counting and ranking in one object. Both are  $O(n)$  with small constants, and both are the kind of thing that makes a script shorter *and* faster at once.

---

## 33.3 Stacks and queues

Two structures with one rule each.

- A **stack** is last in, first out. Undo history, the call stack behind recursion, depth-first search.
- A **queue** is first in, first out. Job queues, retry backlogs, breadth-first search.

A Python list is already a fine stack: `append` and `pop` are both  $O(1)$ . As a queue it is not, because `pop(0)` has to shift every remaining element along:

```
# terminal, in companion/ch33 (from ch33_timings.py, 100,000 items)
list.pop(0)      : 0.873 s
deque.popleft()  : 0.003 s
```

`collections.deque` is a double-ended queue:  $O(1)$  at both ends.

```
from collections import deque

# a stack: the last invoice queried is the first one answered
stack = []
for invoice_id in ["INV000001", "INV000002", "INV000003"]:
    stack.append(invoice_id)
print("stack pops:", [stack.pop() for _ in range(3)])

# a queue: invoices are worked in the order they arrived
queue = deque(["INV000001", "INV000002", "INV000003"])
print("queue pops:", [queue.popleft() for _ in range(3)])

# a retry queue, which is where analysts meet this first
retries = deque([("INV000010", 1), ("INV000011", 1)])
processed = []
while retries:
    invoice_id, attempt = retries.popleft()
    if attempt < 3 and invoice_id.endswith("0"):
        retries.append((invoice_id, attempt + 1))      # try again later, not immediately
    else:
        processed.append((invoice_id, attempt))
print("processed:", processed)
```

```
stack pops: ['INV000003', 'INV000002', 'INV000001']
queue pops: ['INV000001', 'INV000002', 'INV000003']
processed: [('INV000011', 1), ('INV000010', 3)]
```

That last pattern is Chapter 29's retry logic in miniature: failures go to the back of the queue, so one stubborn item doesn't block the rest.

---

## 33.4 Trees and graphs

---

A **tree** is data where each item has one parent: Riverstone's org chart, its bill of materials, a folder structure, a category hierarchy. A **graph** is the general case, where anything can connect to anything: customers and the orders they share, tables and their dependencies (Chapter 32's DAG is a graph), roads between cities.

Chapter 28 walked these in SQL with recursive CTEs. In Python they're a dictionary of lists:

```

reports_to = {
    "Arvind Kapoor": ["Anita Rao", "Harpreet Sethi"],
    "Anita Rao": ["Vikram Singh", "Farah Khan"],
    "Harpreet Sethi": ["Ramesh Patil"],
    "Vikram Singh": ["Neha Kulkarni", "Rahul Mehta"],
    "Ramesh Patil": ["Ajay Kumar"],
}

def depth_first(person, level=0, seen=None):
    seen = seen if seen is not None else set()
    if person in seen:
        return [] # a cycle: stop rather than loop forever
    seen.add(person)
    result = [(level, person)]
    for report in reports_to.get(person, []):
        result += depth_first(report, level + 1, seen)
    return result

for level, person in depth_first("Arvind Kapoor"):
    print("    " * level + person)

```

```

Arvind Kapoor
  Anita Rao
    Vikram Singh
      Neha Kulkarni
      Rahul Mehta
    Farah Khan
  Harpreet Sethi
    Ramesh Patil
      Ajay Kumar

```

Depth-first goes all the way down one branch before starting the next, which is why the output reads like an indented org chart. Breadth-first goes level by level, which is what you want for "who is two levels below the MD?" and for shortest paths:

```

from collections import deque

def breadth_first(start):
    queue, seen, levels = deque([(start, 0)]), {start}, []
    while queue:
        person, level = queue.popleft()
        levels.append((level, person))
        for report in reports_to.get(person, []):
            if report not in seen:
                seen.add(report)
                queue.append((report, level + 1))
    return levels

for level in range(3):
    people = [p for lvl, p in breadth_first("Arvind Kapoor") if lvl == level]
    print(f"level {level}: {' '.join(people)}")

```

```

level 0: Arvind Kapoor
level 1: Anita Rao, Harpreet Sethi
level 2: Vikram Singh, Farah Khan, Ramesh Patil

```

Same tree, two walks: depth-first uses a stack, breadth-first a queue

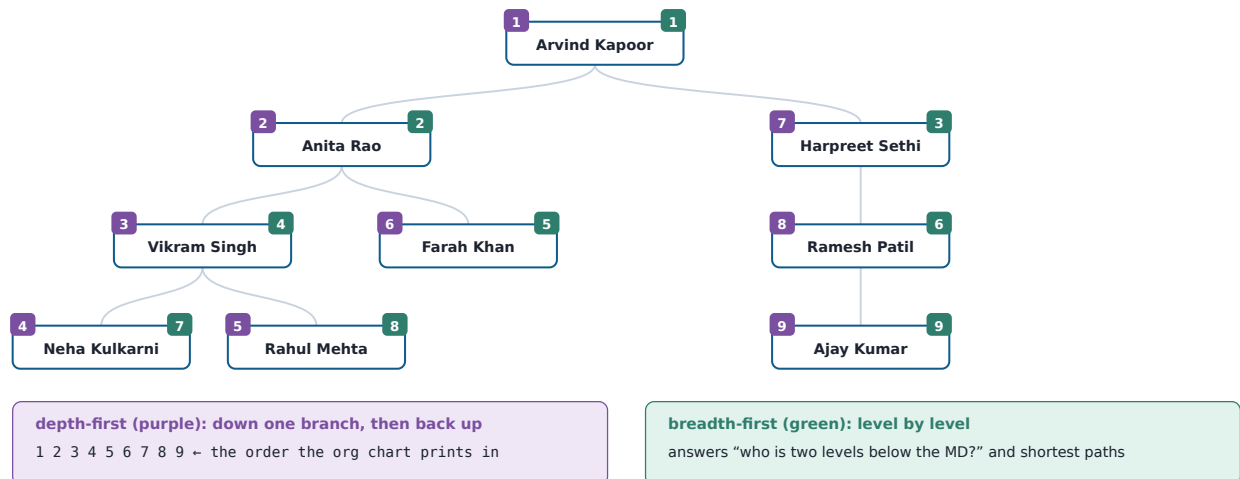


Figure 33.3 – One tree, two orders. The only difference is whether the next node comes off a stack or a queue.

**The two are the same algorithm with a different container.** Depth-first uses a stack (or recursion, which is a stack); breadth-first uses a queue. That single sentence answers a surprising number of interview questions.

## Cycles

Real data has loops in it: an employee whose manager reports to them, a part that contains itself, two dbt models that ref each other. Both walks above carry a `seen` set, which is the standard defense and the Python version of Chapter 28's `CYCLE` clause.

```
broken = {"A": ["B"], "B": ["C"], "C": ["A"]}      # A -> B -> C -> A

def has_cycle(graph):
    def visit(node, path):
        if node in path:
            return True
        for neighbor in graph.get(node, []):
            if visit(neighbor, path | {node}):
                return True
        return False
    return any(visit(node, frozenset()) for node in graph)

print("cycle in the org chart:", has_cycle(reports_to))
print("cycle in the broken data:", has_cycle(broken))
```

```
cycle in the org chart: False
cycle in the broken data: True
```

## 33.5 Searching and sorting

### Linear and binary search

Finding a value in an unsorted list means looking at every element until you find it: **linear search**,  $O(n)$ . If the list is *sorted*, you can halve the search space with every comparison: **binary search**,  $O(\log n)$ . A million sorted values take about twenty comparisons.

```
# terminal, in companion/ch33 (from ch33_timings.py, a sorted list of 1,000,000 numbers)
list.index() 200 searches : 0.713 s
bisect      10,000 searches : 0.0061 s
```

Two hundred linear searches took 0.7 seconds; ten thousand binary searches took six milliseconds. Python's `bisect` module does binary search on any sorted sequence:

```
from bisect import bisect_right, insort

thresholds = [0, 10_000, 25_000, 50_000, 100_000] # discount bands, sorted
labels = ["tiny", "small", "medium", "large", "very large"]

def band(amount):
    return labels[bisect_right(thresholds, amount) - 1]

for amount in (4_500, 25_000, 60_000, 250_000):
    print(f"{amount:>8,}: {band(amount)}")

prices = [290, 430, 750]
insort(prices, 620) # keeps the list sorted
print("prices:", prices)
```

```
4,500: tiny
25,000: medium
60,000: large
250,000: very large
prices: [290, 430, 620, 750]
```

Bucketing a value into sorted bands is the everyday use: it's what a `CASE` statement does in SQL, done in log time instead of by checking every band.

### Sorting

`sorted()` and `list.sort()` use **Timsort**, which is  $O(n \log n)$  and unusually fast on data that's already partly ordered, which real data usually is. Two habits are worth having:

```

lines = [
    {"order_id": "100002", "product_id": "104", "net_revenue": "18600"},
    {"order_id": "100001", "product_id": "101", "net_revenue": "38000"},
    {"order_id": "100001", "product_id": "108", "net_revenue": "5800"},
]

by_revenue = sorted(lines, key=lambda row: float(row["net_revenue"]), reverse=True)
print("largest first:", [row["net_revenue"] for row in by_revenue])

by_order_then_product = sorted(lines, key=lambda row: (row["order_id"], row["product_id"]))
print("stable order:", [(r["order_id"], r["product_id"]) for r in by_order_then_product])

print("largest two without sorting everything:",
      [r["net_revenue"] for r in sorted(lines, key=lambda r: float(r["net_revenue"]), reverse=True)
      [:2]])

```

```

largest first: ['38000', '18600', '5800']
stable order: [('100001', '101'), ('100001', '108'), ('100002', '104')]
largest two without sorting everything: ['38000', '18600']

```

- **key=** sorts by a computed value without changing the data, and a tuple key sorts by several columns, exactly like SQL's `ORDER BY a, b` (Chapter 13).
- **Timsort is stable**: rows that tie keep their original order, which is why sorting twice (by the secondary key, then the primary) gives the same answer as one tuple key.
- For "the biggest three of a million", `heapq.nlargest(3, items, key=...)` is  $O(n \log 3)$  and beats sorting everything.

**When sorting is the algorithm.** Many problems get easy once the data is in order: finding duplicates (equal values sit together), finding the nearest match (binary search), merging two feeds (walk both in step), and grouping (Chapter 13's `GROUP BY` is a sort or a hash under the covers). Paying  $O(n \log n)$  once to make everything after it linear is one of the most common good trades in data work.

## 33.6 Recursion

A **recursive** function calls itself on a smaller version of the problem. It needs two things: a **base case** that stops, and a **recursive case** that moves toward it.

```

parts = {
    # Chapter 28's bill of materials, as a dictionary
    "Garden Chair": [("Seat shell", 1), ("Steel frame", 1), ("Product label", 1), ("Shipping
carton", 1)],
    "Seat shell": [("PP granules", 2.2), ("Masterbatch", 0.08)],
    "Steel frame": [("Leg assembly", 2), ("Steel tube", 1.0), ("Fastener pack", 1)],
    "Leg assembly": [("Steel tube", 1.2), ("Fastener pack", 1)],
}

def explode(part, quantity=1.0):
    if part not in parts:
        # base case: a bought material
        return {part: quantity}
    totals = {}
    for child, child_quantity in parts[part]:
        # recursive case: go one level down
        for material, amount in explode(child, quantity * child_quantity).items():
            totals[material] = totals.get(material, 0) + amount
    return totals

for material, amount in sorted(explode("Garden Chair").items()):
    print(f"{material:<16} {amount:>6.2f}")

```

|                 |      |
|-----------------|------|
| Fastener pack   | 3.00 |
| Masterbatch     | 0.08 |
| PP granules     | 2.20 |
| Product label   | 1.00 |
| Shipping carton | 1.00 |
| Steel tube      | 3.40 |

Steel tube comes out at 3.4, matching Chapter 28's SQL exactly, because a recursive CTE and a recursive function are the same idea in two languages.

**The call stack.** Each call waits for the ones it makes, and Python keeps them on a stack. That stack has a limit (1,000 frames by default), so a walk over a deep structure can end in `RecursionError`:

```

import sys

def depth(n):
    return 0 if n == 0 else 1 + depth(n - 1)

print("limit:", sys.getrecursionlimit())
try:
    depth(5000)
except RecursionError as exc:
    print("RecursionError:", exc)
print("iterative version:", sum(1 for _ in range(5000)))

```

```

limit: 1000
RecursionError: maximum recursion depth exceeded
iterative version: 5000

```

**Recursion or a loop?** Use recursion when the data is itself nested: trees, bills of materials, folders, JSON (Chapter 14), nested comments. Use a loop when the problem is a straight sequence. Any recursion can be rewritten as a loop with an explicit stack, and sometimes should be, which is exactly what section 33.4's breadth-first search did.

## 33.7 Hashing, and why dictionaries are instant

A **hash function** turns a value into a number. Python's `hash()` does it for any immutable object:

```
for value in (100001, 100002, (100001, 101)):
    print(f"{str(value):<20} hash {hash(value):>22} -> bucket {hash(value) % 8}")

print("same value, same hash:", hash((100001, 101)) == hash((100001, 101)))
print("lists are unhashable, which is why they can't be dictionary keys:")
try:
    hash([100001, 101])
except TypeError as exc:
    print(" TypeError:", exc)
```

```
100001          hash          100001 -> bucket 1
100002          hash          100002 -> bucket 2
(100001, 101)   hash  -5250647048266057633 -> bucket 7
same value, same hash: True
lists are unhashable, which is why they can't be dictionary keys:
TypeError: unhashable type: 'list'
```

(Strings hash differently in every Python process, deliberately, so the numbers above use integers to stay reproducible. The behavior is the same.)

A **hash table** (which is what a Python `dict` and `set` are) keeps an array of buckets. To store a key it hashes it, takes the remainder by the number of buckets, and puts the entry there. To find a key it does the same arithmetic and looks in one bucket. **That's why lookup doesn't get slower as the dictionary grows:** the work is one hash and one bucket, whatever the size.

**Why a dictionary lookup does not get slower as it grows**

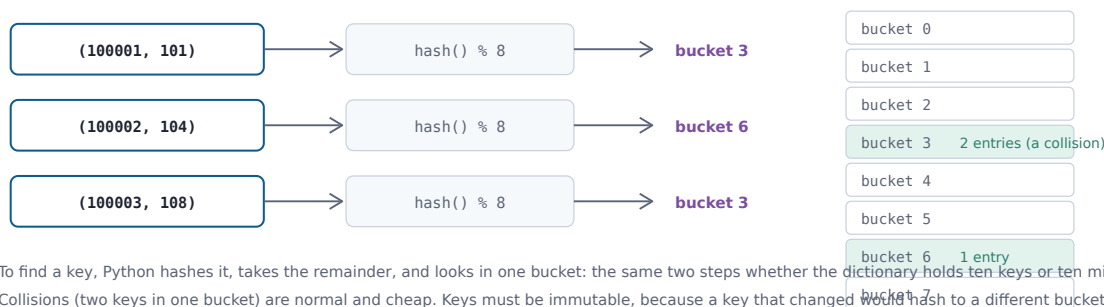


Figure 33.2 – Hash, take the remainder, look in one bucket. Collisions are handled by storing several entries in the same bucket.

Two consequences worth carrying:

- **Keys must be immutable**, because a key that changed after being stored would hash to a different bucket and become invisible. That's why tuples work as keys and lists don't, and why the project's index is keyed by `(order_id, product_id)`.
- **Collisions** (two keys in the same bucket) are normal and handled; they only hurt if almost everything collides, which is a problem for adversarial input, not for order ids.

This is also what Chapter 28's **hash join** does: build a hash table from the smaller table, scan the larger one, and probe the table for each row. When PostgreSQL's plan says `Hash Join`, it is doing what section 33.2's dictionary index did.

---

## 33.8 Memoization, dynamic programming, and greedy

---

Three named ideas you'll meet in interviews and, occasionally, in real work.

### Memoization: remember what you already computed

The classic demonstration is Fibonacci, where the naive recursion recomputes the same values exponentially often:

```
# terminal, in companion/ch33 (from ch33_timings.py)
fib(25) plain      : 0.008 s
fib(30) plain      : 0.087 s
fib(32) plain      : 0.226 s
fib(32) memoized   : 0.000023 s
fib(300) memoized  : 22232244629420445529739893461909967206666939096499764990979600
```

Each step of five in `n` multiplies the plain version's time by about ten: that's exponential growth,  $O(2^n)$ . Caching turns it linear, and Python has the cache built in:

```
from functools import lru_cache

calls = {"plain": 0, "cached": 0}

def fib(n):
    calls["plain"] += 1
    return n if n < 2 else fib(n - 1) + fib(n - 2)

@lru_cache(maxsize=None)
def fib_cached(n):
    calls["cached"] += 1
    return n if n < 2 else fib_cached(n - 1) + fib_cached(n - 2)

fib(25); fib_cached(25)
print(f"fib(25) plain made {calls['plain']:,} calls")
print(f"fib(25) cached made {calls['cached']:,} calls")
print("same answer:", fib(25) == fib_cached(25))
```

```
fib(25) plain made 242,785 calls
fib(25) cached made 26 calls
same answer: True
```

242,785 calls against 26. In analytics work, `@lru_cache` pays off on an expensive pure function called repeatedly with the same arguments: a currency conversion, a geocode, a parsed configuration.

## Dynamic programming

**Dynamic programming** is memoization turned into a table you fill in deliberately: solve the small subproblems once, in order, and build the answer from them. The question it answers is always "the best way to do X given a limit", and it appears in interviews far more often than in dashboards.

```
def fewest_notes(amount, notes=(2000, 500, 200, 100, 50, 20, 10)):
    """The fewest notes that make an amount exactly, or None if it can't be made."""
    best = [0] + [None] * amount
    for value in range(1, amount + 1):
        options = [best[value - note] + 1 for note in notes if note <= value and best[value - note]
is not None]
        best[value] = min(options) if options else None
    return best[amount]

for amount in (760, 2750, 45):
    answer = fewest_notes(amount)
    print(f"₹{amount:>5}: {answer} notes" if answer else f"₹{amount:>5}: cannot be made exactly")
```

```
₹ 760: 4 notes
₹ 2750: 4 notes
₹ 45: cannot be made exactly
```

Every amount below the target is solved once and reused, which is what makes this  $O(\text{amount} \times \text{notes})$  rather than exponential.

## Greedy

A **greedy** algorithm takes the best-looking option at each step and never reconsiders. It's simpler and faster, and it's right only when the problem has the right shape.

```
def greedy_notes(amount, notes=(2000, 500, 200, 100, 50, 20, 10)):
    count = 0
    for note in notes:
        count, amount = count + amount // note, amount % note
    return count if amount == 0 else None

odd_notes = (10, 6, 1)
def greedy_odd(amount, notes=odd_notes):
    count = 0
    for note in notes:
        count, amount = count + amount // note, amount % note
    return count

print("Indian notes, ₹760: greedy", greedy_notes(760), "vs best", fewest_notes(760))
print("made-up notes 10/6/1, 12: greedy", greedy_odd(12), "vs best", fewest_notes(12, odd_notes))
```

```
Indian notes, ₹760: greedy 4 vs best 4
made-up notes 10/6/1, 12: greedy 3 vs best 2
```

With real Indian notes, greedy is optimal. With notes of 10, 6 and 1, greedy takes  $10 + 1 + 1$  (three) where  $6 + 6$  (two) is better. **The lesson isn't about change:** it's that "take the biggest first" is a guess about the shape of the problem, and it needs checking. Greedy algorithms you'll actually meet: assigning leads to the rep with the fewest open ones, packing files into batches, picking the next job by earliest deadline.

## 33.9 Memory, and the other costs

Speed isn't the only resource. A list of a million rows holds a million objects; a **generator** produces them one at a time and holds one.

```
# terminal, in companion/ch33 (from ch33_timings.py, tracemalloc peak)
list      : 40.4 MB peak, sum 999999000000
generator : 0.0005 MB peak, sum 999999000000
```

Same answer, 80,000 times less memory. Chapter 29's API client used the same idea with `yield from` to hand back leads page by page.

```
def line_values(path):
    """Yields one value at a time; nothing is held except the current row."""
    import csv
    with open(path, newline="", encoding="utf-8") as f:
        for row in csv.DictReader(f):
            yield float(row["net_revenue"])

total = sum(line_values("match_data/order_lines.csv"))
biggest = max(line_values("match_data/order_lines.csv"))
print(f"total revenue {total:,.0f}, largest line {biggest:,.0f}")

squares_list = [n * n for n in range(5)]      # a list comprehension: builds everything
squares_gen = (n * n for n in range(5))       # a generator expression: builds nothing yet
print("list:", squares_list)
print("generator:", squares_gen.__class__.__name__, "->", list(squares_gen))
```

```
total revenue 4,421,366,900, largest line 140,000
list: [0, 1, 4, 9, 16]
generator: generator -> [0, 1, 4, 9, 16]
```

### The trade-offs to keep in mind:

| Choice                             | Costs                                        | Saves                                    |
|------------------------------------|----------------------------------------------|------------------------------------------|
| Generator instead of a list        | can only be read once; no <code>len()</code> | memory, and time before the first result |
| A dictionary index                 | memory for the index                         | vastly more time                         |
| Caching ( <code>lru_cache</code> ) | memory, and staleness if inputs change       | repeated computation                     |
| Sorting once                       | $O(n \log n)$ up front                       | linear scans afterwards                  |
| pandas instead of loops            | memory (whole frame) and a dependency        | a lot of time, through compiled code     |

**Watch out: the fastest fix is often not an algorithm at all.** Before rewriting a loop, ask whether the work should be happening in Python. Chapter 28's lesson applies: if the data is in a database, an indexed query moves the join to a system built for it. If it's in a DataFrame, a vectorized pandas operation runs in compiled code. A dictionary index is the right answer when the data is already in Python and has to stay there.

## 33.10 What coding interviews actually test

Analytics engineering and data engineering interviews include a coding round, usually 30 to 45 minutes on one or two problems. They are not testing whether you memorized quicksort.

### What they're looking for:

| They want to see              | How to show it                                                                        |
|-------------------------------|---------------------------------------------------------------------------------------|
| You clarify before coding     | ask about size, duplicates, nulls, ordering, and what to return when nothing matches  |
| You pick a sensible structure | "I'll index the orders by id in a dictionary, so the lookup is constant time"         |
| You can state the complexity  | "one pass to build the index, one pass over invoices: $O(n + m)$ time, $O(n)$ memory" |
| You write code that runs      | small, named helpers; no clever one-liners                                            |
| You test it                   | one normal case, one empty, one duplicate, one no-match                               |
| You improve it when prompted  | "if the file didn't fit in memory, I'd stream it and index only the keys"             |

**The patterns that come up most**, all of which are in this chapter: a hash map to avoid a nested loop; two pointers or a merge over sorted data; a set for deduplication; a counter for frequency; breadth-first or depth-first over a tree or graph; a sort followed by a linear pass.

**The sentence that earns marks:** after your first working version, say *"this is  $O(n^2)$  because of the inner loop; I can make it  $O(n)$  with a dictionary"*, then do it. Interviewers care much more about that than about arriving at the optimal answer first.

Chapter 72's Python and pandas bank has the questions themselves, and Chapter 69 covers how to talk through a problem under pressure.

## Common mistakes and how to spot them

| Mistake                                              | Symptom                                                | Fix                                                             |
|------------------------------------------------------|--------------------------------------------------------|-----------------------------------------------------------------|
| A lookup inside a loop ( <code>x in a_list</code> )  | The script slows down as the data grows, with no error | Build a <code>set</code> or <code>dict</code> before the loop   |
| A loop inside a loop over the same data              | Fine on a sample, hours on the real file               | Index one side, then one pass over the other                    |
| <code>list.pop(0)</code> as a queue                  | Quietly quadratic                                      | <code>collections.deque</code>                                  |
| Re-reading or re-parsing the same file inside a loop | Disk time dominates everything                         | Read once into a structure                                      |
| Recursion with no base case, or on cyclic data       | <code>RecursionError</code> , or a hang                | A base case, a <code>seen</code> set, and a depth limit         |
| Deep recursion on big structures                     | <code>RecursionError</code> at 1,000 frames            | Rewrite with an explicit stack, or raise the limit deliberately |

| Mistake                                                  | Symptom                                 | Fix                                                                                 |
|----------------------------------------------------------|-----------------------------------------|-------------------------------------------------------------------------------------|
| Mutable objects as dictionary keys                       | <code>TypeError: unhashable type</code> | Use tuples                                                                          |
| Sorting inside a loop                                    | $O(n^2 \log n)$ by accident             | Sort once, before the loop                                                          |
| <code>sorted(...)[:3]</code> on millions of rows         | Sorting everything to keep three        | <code>heapq.nlargest</code>                                                         |
| Reading a huge file into a list                          | The process is killed for memory        | A generator, or read in chunks                                                      |
| Caching a function whose inputs change                   | Stale results nobody can explain        | Cache pure functions only; clear or bound the cache                                 |
| Optimizing the wrong line                                | Hours spent, no change                  | Measure first: <code>time.perf_counter</code> , <code>timeit</code> , or a profiler |
| Reaching for an algorithm when the database should do it | A Python join over a million rows       | Let SQL join, with the index from Chapter 28                                        |
| Quoting Big-O as if it were time                         | "It's $O(n)$ , so it's fast"            | Big-O is growth; the stopwatch is speed                                             |

## In the real world: the invoice match that ran all afternoon

Riverstone's accounts team gets one file a month from its largest supplier: every line the supplier says it delivered. Somebody has to check each line against the order it bills. Priya wrote the script two years ago, and it worked: twenty minutes, run over lunch.

This month it hasn't finished by four o'clock, and the month-end close needs it.

Meera reads the script. It does exactly what a careful person would write: for each invoice line, loop through the order lines until you find a match. Two years ago that was 3,000 invoices against 40,000 order lines. Today it's 20,000 against 176,000, and the work is the product of the two: **thirty times the invoices, four times the lines, a hundred and twenty times the work.**

She times the current version on a slice, rather than guessing: 2,000 invoices take 7.5 seconds, so the full file is about 75 seconds of matching... and the script had been running for six hours. The nested loop wasn't the whole story. The script also re-read the order file *inside* the loop, so the real cost was 20,000 file reads.

The fix takes twenty minutes:

1. **Read each file once.** That alone takes the job from hours to about a minute and a half.
2. **Build a dictionary** keyed by `(order_id, product_id)` and look each invoice up: 0.045 seconds to build, 0.009 seconds to match all 20,000.
3. **A set of already-processed invoice ids**, so a re-sent file doesn't double-count.
4. **A generator** for reading, so next year's bigger file doesn't have to fit in memory.

The final run takes **0.07 seconds** of matching, inside a script that spends most of its time reading the files. Meera adds Chapter 29's tests (the counts, and one hand-checked match), and a line to the log saying how many matched and how many didn't, because the 500 unmatched invoices are the actual

business output: those are the ones the supplier has to explain.

**What she tells the accounts manager:** *"It wasn't broken, it was written for a smaller file. It now finishes before you've finished reading the email, and it reports the 500 lines that don't match anything, which is the part you actually need."*

Notice the order she worked in: measure, find the real cost, fix the structure, then test. The clever algorithm was a dictionary, which is the first thing a Python programmer learns.

---

## Tools

---

Versions used for this chapter, checked in September 2026:

- **Python 3.12.3**, standard library only: `collections` (`deque`, `Counter`, `defaultdict`), `bisect`, `heapq`, `functools` (`lru_cache`), `itertools`, `time`, `timeit`, `tracemalloc`, `sys`.
- **Measuring:** `time.perf_counter()` for whole runs, `timeit` for small operations, `tracemalloc` for memory, and `cProfile` with `snakeviz` or `py-spy` when you need to find the slow line rather than confirm it.
- **When Python isn't the answer:** `pandas` and `NumPy` (Chapter 17) for vectorized work, the database (Chapter 28) for joins and aggregation over large tables, and `Polars` or `DuckDB` when a single machine has to chew through files.
- **Companion files** in `ch33/`: `generate_ch33_data.py` (176,110 order lines and 20,000 invoice lines, seed 33), `match_slow.py` and `match_fast.py` (the project's before and after), and `ch33_timings.py`, which reproduces every timing in the chapter.

**Tool note: profile before you optimize.** `python3 -m cProfile -s cumtime script.py | head -20` names the functions where the time actually goes. Analysts routinely discover that 90% of a "slow algorithm" is CSV parsing, date conversion, or a database round trip in a loop. The algorithm is what's left after the measurement, not before it.

---

## The project: make the matching script fast, and prove it

---

**Goal:** take a working but slow script, make it fast by changing structures rather than cleverness, and measure every step.

**Option A: your own slow script.** Anything that loops over rows and looks something up. Time it before you touch it.

**Option B: Riverstone.** Start from `match_slow.py` and rebuild it yourself before comparing with `match_fast.py`.

**Steps:**

1. **Measure first.** Time the original on 250, 500, 1,000 and 2,000 invoices. Plot or tabulate the four numbers and say what shape they are.
2. **Predict.** From those numbers, how long would the full 20,000-line file take? Write the prediction down before running it.
3. **Find the real cost.** Profile it ( `cProfile` ). Is the time in the comparison, in parsing, or in reading the file?
4. **Index it.** Build a dictionary keyed by ( `order_id`, `product_id` ), and rerun. Report the build time and the match time separately.
5. **Dedupe with a set.** Ignore invoice lines you have already processed, and prove it works by running the file twice.
6. **Stream it.** Convert the reader to a generator so memory stays flat, and check with `tracemalloc` .
7. **Reconcile.** Matched plus unmatched must equal the invoice count, and the totals must agree with the slow version's. Write those as tests (Chapter 29).
8. **Report both numbers:** the time, and the 500 unmatched invoice lines, which is what the business wanted all along.

**Deliverables:** the before-and-after scripts, a table of timings at each step, the tests, and three sentences on which change mattered most.

#### Stretch goals:

- Match on ( `order_id`, `product_id` ) **and** an amount tolerance of 2%, without making the lookup linear again.
- Handle one order line billed across two invoices: what has to change, and what does the grain become?
- Do the same job in SQL ( `LEFT JOIN` , Chapter 28) and in pandas ( `merge` , Chapter 17), and time all three.
- Make the script handle a 5 GB file on a laptop with 8 GB of memory.

---

## You've got it when...

- ☐ I can name the complexity of a loop I wrote, and say what happens when the data doubles.
- ☐ I reach for a dictionary or a set the moment I see a lookup inside a loop.
- ☐ I can say why `x in a_set` is instant and `x in a_list` isn't.
- ☐ I use `deque` for queues and know why `pop(0)` is a trap.
- ☐ I can walk a tree depth-first and breadth-first, and guard against cycles.
- ☐ I know what `sorted(key=...)` costs and when sorting first makes everything easier.
- ☐ I can write a recursive function with a base case, and say when a loop would be clearer.
- ☐ I can explain a hash table, and connect it to a database hash join.
- ☐ I recognize memoization, dynamic programming, and greedy, and know greedy needs checking.

- ☐ I think about memory as well as time, and use generators for big files.
  - ☐ I measure before optimizing, and state complexity out loud in interviews.
- 

## Recap

---

- **Big-O** describes how work grows:  $O(1)$ ,  $O(\log n)$ ,  $O(n)$ ,  $O(n \log n)$ ,  $O(n^2)$ ,  $O(2^n)$ . It's a shape, not a stopwatch, and quadratic is the shape that kills scripts.
  - **Lists** keep order; **dictionaries** and **sets** find things in constant time; **tuples** are immutable and can be keys. The single most valuable habit in this chapter: **build an index before the loop**, which is a hand-written hash join.
  - **Stacks** are last in, first out; **queues** are first in, first out; use `collections.deque`, not `list.pop(0)`.
  - **Trees and graphs** are dictionaries of lists. **Depth-first** uses a stack, **breadth-first** uses a queue, and both need a `seen` set on real data.
  - **Binary search** (`bisect`) is  $O(\log n)$  on sorted data; **sorting** is  $O(n \log n)$  and often makes everything after it easy. `heapq.nlargest` beats sorting for a top few.
  - **Recursion** needs a base case; Python's stack limit is 1,000 frames; use it for nested data, a loop for sequences.
  - **Hashing** is why dictionary lookups don't slow down, why keys must be immutable, and how database hash joins work.
  - **Memoization** (`lru_cache`) turns repeated work into a lookup; **dynamic programming** fills a table of subproblems; **greedy** takes the best local step and is only sometimes right.
  - **Memory** matters too: generators hold one row, lists hold all of them.
  - **Measure first**. Profile, fix the real cost, and remember that the database or pandas may be the right answer instead of any algorithm.
- 

## Practice exercises

---

Work in `companion/ch33`, with the data built by `generate_ch33_data.py`. Predict each answer before running it.

### Warm-up

1. What is the complexity of each: (a) summing a list; (b) checking `x in a_dict`; (c) `sorted(rows)`; (d) a loop over invoices with `if invoice_id in processed_list` inside?
2. Count the distinct `(order_id, product_id)` pairs in the order lines file, twice: once with a list of seen pairs, once with a set. Time both on the first 20,000 rows.
3. Build a dictionary of order lines by `order_id` (not by the pair). How many orders are there, and what is the largest number of lines on one order?
4. Use `Counter` to find the three products that appear on the most invoice lines.

## Core

5. Rewrite this quadratic snippet to run in linear time, and prove the results are identical: `[inv for inv in invoices if any(l["order_id"] == inv["order_id"] for l in order_lines)]`.
6. Write `band(amount)` for Riverstone's discount tiers with `bisect`, and test it on the boundary values exactly.
7. Walk the bill of materials in section 33.6 iteratively, with an explicit stack instead of recursion. Confirm you get the same quantities.
8. Find the 10 largest order lines by revenue with `sorted` and with `heapq.nlargest`, and time both on all 176,110 rows.
9. Detect whether a dictionary of dbt-style model dependencies contains a cycle, and report the cycle's members, not just that one exists.
10. Write a generator that yields only order lines above ₹50,000 from the file, and use it to compute the count and total without holding the file in memory.

## Stretch

11. Match invoices to order lines where the amount must be within 2%, without a nested loop. What does the index have to hold?
12. Implement binary search yourself (no `bisect`) and test it against `bisect_left` on 1,000 random cases, including values that aren't present.
13. Memoize a function that parses dates from strings, and measure the difference on a file with many repeated dates.
14. Use `cProfile` on `match_slow.py` with 500 invoices. Which function has the highest cumulative time, and is it the one you expected?
15. Write the fewest-notes function from section 33.8 using recursion with `lru_cache` instead of a table. Which version would you rather maintain, and are they the same complexity?

## Think about it (no code needed)

16. A colleague says "we'll just add more memory" to a script that is  $O(n^2)$  in time. What do you say?
17. Your script does a database query inside a loop over 5,000 customers. Name three fixes, in order of how much they'd help.
18. When is an  $O(n^2)$  algorithm perfectly acceptable?

---

## Key terms

algorithm · complexity · Big-O · constant time · logarithmic time · linear time · linearithmic · quadratic · exponential · data structure · list · dictionary · set · tuple · immutability · index (in-memory) · hash function · hash table · bucket · collision · hash join · stack · queue · `deque` · last in first out · first in first out · tree · graph · node · edge · depth-first search · breadth-first search · cycle detection · linear search · binary search · `bisect` · sorting · Timsort · stable sort · sort key · heap · `heapq` · recursion ·

base case · recursive case · call stack · recursion limit · memoization · `lru_cache` · dynamic programming · subproblem · greedy algorithm · generator · lazy evaluation · profiling · `cProfile` · `timeit` · `tracemalloc` · vectorization

(All terms are defined in the Glossary, Appendix A.)

---

## Where this leads

---

- **Chapter 28, Advanced SQL, Performance & Data Modeling**, is the database version of this chapter: indexes are hash tables and B-trees, and a hash join is section 33.2's dictionary.
  - **Chapter 29, Python as Software, Not Scripts**, is how a fast script becomes a maintained one.
  - **Chapter 17, Python for Data Analysis**, covers the vectorized alternatives that often beat any loop you could write.
  - **Chapter 46, Pipelines & Orchestration**, and **Chapter 49, Storage, Warehouses & Lakehouses**, are where these costs meet distributed systems and cloud bills.
  - **Chapter 72, Python & pandas Question Bank**, has the coding questions; **Chapter 69** covers how to talk through them.
- 

## Answers to practice exercises

---

(In the finished book these move to Appendix G.)

1. (a)  $O(n)$ : every element is added. (b)  $O(1)$ : one hash, one bucket. (c)  $O(n \log n)$ : Timsort. (d)  $O(n \times m)$ : the `in` on a list is itself a scan, so the whole loop is quadratic. Changing `processed_list` to a set makes it  $O(n)$ , which is exercise 2's point.

2.

```

import csv, time

def load(path):
    with open(path, newline="", encoding="utf-8") as f:
        return list(csv.DictReader(f))

rows = load("match_data/order_lines.csv")[:20_000]

def with_list(rows):
    seen = []
    for row in rows:
        key = (row["order_id"], row["product_id"])
        if key not in seen:
            seen.append(key)
    return seen

def with_set(rows):
    return {(row["order_id"], row["product_id"]) for row in rows}

list_time = time.perf_counter()
pairs_list = with_list(rows)
list_time = time.perf_counter() - list_time
set_time = time.perf_counter()
pairs_set = with_set(rows)
set_time = time.perf_counter() - set_time

print(f"list: {len(pairs_list):,} pairs")
print(f"set: {len(pairs_set):,} pairs")
print("same answer:", set(pairs_list) == pairs_set)
print("the set was faster by a factor of:", "hundreds" if set_time * 100 < list_time else "less than 100")

```

```

list: 20,000 pairs
set: 20,000 pairs
same answer: True
the set was faster by a factor of: hundreds

```

Same answer, and on the test machine the set version was about 650 times faster on these 20,000 rows. The gap grows with the data, because the list version is  $O(n^2)$ : each `not in` scans everything found so far. On the full 176,110 rows it would take minutes; the set takes a fraction of a second.

### 3.

```

from collections import defaultdict

all_rows = load("match_data/order_lines.csv")
by_order = defaultdict(list)
for row in all_rows:
    by_order[row["order_id"]].append(row)

largest = max(by_order, key=lambda order_id: len(by_order[order_id]))
print(f"{len(by_order):,} orders")
print(f"largest order {largest} has {len(by_order[largest])} lines")
print("lines per order:", {n: sum(1 for v in by_order.values() if len(v) == n) for n in (1, 2, 3)})

```

```

66,667 orders
largest order 100001 has 3 lines
lines per order: {1: 1003, 2: 21885, 3: 43779}

```

Grouping into a dictionary of lists is the Python version of `GROUP BY`, and it's one pass. Note the grain (Chapter 28): one line per order and product, so the biggest order has as many lines as there are products.

4.

```
from collections import Counter

invoices = load("match_data/supplier_invoices.csv")
print(Counter(invoice["product_id"] for invoice in invoices).most_common(3))
```

```
[('101', 2607), ('107', 2561), ('104', 2514)]
```

5.

```
order_ids = {line["order_id"] for line in all_rows}          # one pass, a set
linear = [inv for inv in invoices if inv["order_id"] in order_ids]

quadratic_sample = [inv for inv in invoices[:200]
                    if any(l["order_id"] == inv["order_id"] for l in all_rows)]
print(f"linear version kept {len(linear):,} of {len(invoices):,} invoices")
print("same answer on the first 200:",
      [i["invoice_line_id"] for i in linear if i in invoices[:200]] == [i["invoice_line_id"] for i
in quadratic_sample])
```

```
linear version kept 19,500 of 20,000 invoices
same answer on the first 200: True
```

The quadratic version is only run on 200 invoices here because the full version would take about a minute; that's the comparison worth internalizing.

6.

```
from bisect import bisect_right

thresholds = [0, 10_000, 25_000, 50_000, 100_000]
labels = ["tiny", "small", "medium", "large", "very large"]

def band(amount):
    return labels[bisect_right(thresholds, amount) - 1]

for amount in (0, 9_999, 10_000, 24_999, 25_000, 100_000, 100_001):
    print(f"{amount:>8,}: {band(amount)}")
```

```
0: tiny
9,999: tiny
10,000: small
24,999: small
25,000: medium
100,000: very large
100,001: very large
```

Boundaries are where banding bugs live: `bisect_right` puts a value *equal* to a threshold into the higher band, which matches "₹10,000 and above". If the rule is "over ₹10,000", `bisect_left` is the one you want. Write the test with the boundary values, not with 9,500 and 30,000.

7.

```
parts = {
    "Garden Chair": [("Seat shell", 1), ("Steel frame", 1), ("Product label", 1), ("Shipping carton", 1)],
    "Seat shell": [("PP granules", 2.2), ("Masterbatch", 0.08)],
    "Steel frame": [("Leg assembly", 2), ("Steel tube", 1.0), ("Fastener pack", 1)],
    "Leg assembly": [("Steel tube", 1.2), ("Fastener pack", 1)],
}

def explode_iterative(part, quantity=1.0):
    totals, stack = {}, [(part, quantity)]
    while stack:
        current, current_quantity = stack.pop()
        if current not in parts:
            totals[current] = totals.get(current, 0) + current_quantity
            continue
        for child, child_quantity in parts[current]:
            stack.append((child, current_quantity * child_quantity))
    return totals

for material, amount in sorted(explode_iterative("Garden Chair").items()):
    print(f"{material:<16} {amount:>6.2f}")
```

|                 |      |
|-----------------|------|
| Fastener pack   | 3.00 |
| Masterbatch     | 0.08 |
| PP granules     | 2.20 |
| Product label   | 1.00 |
| Shipping carton | 1.00 |
| Steel tube      | 3.40 |

Identical to the recursive version, because the stack is what recursion was using. The iterative form has no depth limit, which matters for deep structures; the recursive form is shorter to read. Choose per structure, and say which you chose in a code review.

8.

```
import heapq, time

values = [(float(row["net_revenue"]), row["order_id"]) for row in all_rows]

started = time.perf_counter()
top_sorted = sorted(values, reverse=True)[:10]
sort_time = time.perf_counter() - started

started = time.perf_counter()
top_heap = heapq.nlargest(10, values)
heap_time = time.perf_counter() - started

print("same answer:", top_sorted == top_heap)
print("largest line:", top_sorted[0])
print("nlargest was quicker:", heap_time < sort_time)
```

```
same answer: True
largest line: (140000.0, '166653')
nlargest was quicker: True
```

On the test machine that was 0.050 seconds against 0.006. `sorted` orders all 176,110 values to keep 10; `nlargest` keeps a heap of 10 and scans once. The gap widens as the data grows and narrows as  $k$  approaches  $n$ ; for  $k$  near  $n$ , just sort.

## 9.

```
models = {"stg_orders": [], "fct_sales": ["stg_orders", "dim_customer"],
          "dim_customer": ["snapshot"], "snapshot": ["fct_sales"]}

def find_cycle(graph):
    def visit(node, path):
        if node in path:
            return path[path.index(node):] + [node]
        for neighbor in graph.get(node, []):
            found = visit(neighbor, path + [node])
            if found:
                return found
        return None
    for node in graph:
        found = visit(node, [])
        if found:
            return found
    return None

print("cycle:", " -> ".join(find_cycle(models)))
```

```
cycle: fct_sales -> dim_customer -> snapshot -> fct_sales
```

Carrying the path rather than a bare `seen` set is what lets you report the members. This is what dbt does when it refuses to run a project with a circular `ref` (Chapter 32), and what Chapter 28's `CYCLE` clause does in SQL.

## 10.

```
def big_lines(path, floor=50_000):
    with open(path, newline="", encoding="utf-8") as f:
        for row in csv.DictReader(f):
            value = float(row["net_revenue"])
            if value > floor:
                yield value

count = total = 0
for value in big_lines("match_data/order_lines.csv"):
    count += 1
    total += value
print(f"{count:,} lines above 50,000, worth {total:,.0f}")
```

```
25,133 lines above 50,000, worth 2,175,373,500
```

One row is in memory at a time, so this works on a file of any size. The trade: the generator can be read only once, so computing both a count and a maximum means either two passes or accumulating both in one loop, as here.

**11.** The index can't be keyed on the amount, because the tolerance makes it a range rather than a value. Key it on `(order_id, product_id)` as before, and check the tolerance *after* the lookup:

```
line = index.get((invoice["order_id"], invoice["product_id"]))
if line is not None and abs(float(invoice["amount"]) - float(line["net_revenue"])) <= 0.02 *
float(line["net_revenue"]):
    ...
```

The lookup stays  $O(1)$  and the tolerance is one comparison. If you had to match on amount *alone*, sort the lines by amount once and use `bisect` to find the window, which is  $O(\log n)$  per invoice: the general lesson is that exact matches want a hash, and range matches want sorted data.

**12.**

```
from bisect import bisect_left
import random

def my_bisect_left(values, target):
    low, high = 0, len(values)
    while low < high:
        middle = (low + high) // 2
        if values[middle] < target:
            low = middle + 1
        else:
            high = middle
    return low

numbers = sorted(random.Random(33).sample(range(1_000_000), 5_000))
tests = [random.Random(34).randrange(1_000_000) for _ in range(1_000)]
print("matches bisect_left on every case:",
      all(my_bisect_left(numbers, t) == bisect_left(numbers, t) for t in tests))
print("example:", my_bisect_left(numbers, numbers[100]), bisect_left(numbers, numbers[100]))
```

```
matches bisect_left on every case: True
example: 100 100
```

The two traps when writing this by hand are the loop condition (`<` against `<=`) and the update (`middle + 1` against `middle`), which is why the test includes values that aren't in the list.

**13.** Wrap the parser in `@lru_cache` and call it on a column with many repeats:

```

from datetime import date
from functools import lru_cache

@lru_cache(maxsize=None)
def parse_date(text):
    year, month, day = (int(part) for part in text.split("-"))
    return date(year, month, day)

dates = [row["order_date"] for row in all_rows]
parsed = [parse_date(text) for text in dates]
info = parse_date.cache_info()
print(f"{len(dates):,} values, {len(set(dates)):,} distinct")
print(f"cache hits {info.hits:,}, misses {info.misses:,}")
print("earliest:", min(parsed), "latest:", max(parsed))

```

```

176,110 values, 700 distinct
cache hits 175,410, misses 700
earliest: 2024-01-01 latest: 2025-11-30

```

700 distinct dates in 176,110 rows, so the cache answers 99.6% of the calls. This is the shape of case where `lru_cache` is worth it: a pure function, expensive relative to a dictionary lookup, called repeatedly with few distinct inputs. It would be worth nothing if every value were unique.

**14.** Run `python3 -m cProfile -s cumtime match_slow.py 500 | head -15`. The highest cumulative time is `match`, as expected, but look at the next lines: on this data the comparisons themselves dominate, while `load` and `csv.DictReader` account for a fraction of the run. That's the useful outcome either way: profiling either confirms your suspicion (fix the algorithm) or overturns it (fix the parsing, the file reading, or the database calls), and guessing gets it wrong about half the time.

**15.**

```

from functools import lru_cache

NOTES = (2000, 500, 200, 100, 50, 20, 10)

@lru_cache(maxsize=None)
def fewest_recursive(amount):
    if amount == 0:
        return 0
    options = [fewest_recursive(amount - note) for note in NOTES if note <= amount]
    options = [o + 1 for o in options if o is not None]
    return min(options) if options else None

for amount in (760, 2750, 45):
    print(f"₹{amount:>5}: {fewest_recursive(amount)}")

```

```

₹ 760: 4
₹ 2750: 4
₹ 45: None

```

Same answers, same complexity: with the cache, each amount is solved once, which is precisely what the table version did. The recursive version reads closer to the definition of the problem; the table version has no recursion limit to worry about and is easier to reason about for large amounts. Most teams take the recursive one for clarity and add a comment saying why the cache is there.

**16.** Memory isn't the constraint; time is. Doubling the memory does nothing for an algorithm whose work is quadratic, and the machine will keep getting slower every month as the data grows. Say it in terms of growth: *"this takes four times as long each time our data doubles, so next year it's four hours, not two. Changing the lookup to a dictionary makes it linear, and it's twenty minutes of work."* If it really is memory-bound rather than time-bound, that's a different conversation, and a generator is usually cheaper than more hardware.

**17.** In order: (1) **Do it in one query.** Fetch all 5,000 customers' rows in a single statement with `WHERE customer_id IN (...)` or a join, then index the result in Python. That removes 4,999 round trips. (2) **Push the work into SQL entirely.** If the loop is computing something the database could compute, let it (Chapter 28). (3) **If the loop must stay, batch it:** 50 queries of 100 ids beat 5,000 of one. A fourth, only if the first three aren't available: cache repeated lookups. Network round trips inside loops are the single most common performance bug in analyst code, and they don't show up in a Big-O count at all.

**18.** When  $n$  is small and stays small: ranking ten regions, comparing eight products, reconciling two dozen accounts. When it runs once, by hand, and finishing in a minute is fine. When the quadratic version is plainly correct and the linear one would need an index that complicates the code for a file that will never grow. The judgment is about  **$n$  and its future**, not about the notation: write the simple version, measure it on real data, and change it when the measurement says so.